

Learning to Pour using Deep Deterministic Policy Gradients

Chau Do*

Camilo Gordillo*

Wolfram Burgard

Abstract—Pouring is a fundamental skill for robots in both domestic and industrial environments. Ideally, a robot should be able to pour with high accuracy to specific, pre-defined heights and without spilling. However, due to the complex dynamics of liquids, it is difficult to learn how to pour to achieve these goals. In this paper we present an approach to learn a policy for pouring using Deep Deterministic Policy Gradients (DDPG). We remove the need for collecting training experiences on a real robot, by using a state-of-the-art liquid simulator, which allows for learning the liquid dynamics. We show through our experiments, performed with a PR2 robot, that it is possible to successfully transfer the learned policy to a real robot and even apply it to different liquids.

I. INTRODUCTION

Pouring is an essential skill for robots performing tasks in both domestic and industrial environments. In particular, robots should be able to pour liquids up to specific, pre-defined heights, without spilling. To achieve this task, the robot must execute a pouring trajectory that takes into consideration different aspects including the position of the containers and the amount of liquid already poured. Overall, pouring is typically regarded as a challenging problem in particular because of the complex liquid dynamics.

In the past, various approaches have been applied to learning how to pour, ranging from learning from demonstration to reinforcement learning. These works mainly focused on collision and spillage avoidance while typically pouring all the contents of a container. In this paper, we focus on the more difficult problem of learning how to pour a specific amount of liquid without spillage or collision.

In order to tackle this problem, we apply recent techniques from deep reinforcement learning. Specifically we use Deep Deterministic Policy Gradients (DDPG) [14] and goal directed behavior for learning the dynamics and purpose of the task. Due to the large amount of training samples required and the potential damage to the robot, it is impractical to train the entire policy on a real robot. Therefore, we perform training using the state-of-the-art liquid simulator *PreonLab* [6]. Through the usage of the simulator, it is possible to learn complex liquid dynamics. Using recent domain transfer techniques, we are able to apply a policy learned in simulation to a real robot. Additionally, the learned policy can be applied to pour different types of liquids. Figure 1 shows our PR2 robot conducting a pouring experiment with the learned policy.

* These authors contributed equally.

This work has been partly supported by BrainLinks-BrainTools, Cluster of Excellence funded by the German Research Foundation (DFG, grant number EXC 1086).

All authors are with the Department of Computer Science at the University of Freiburg, Georges-Köhler-Allee 80, 79110 Freiburg, Germany

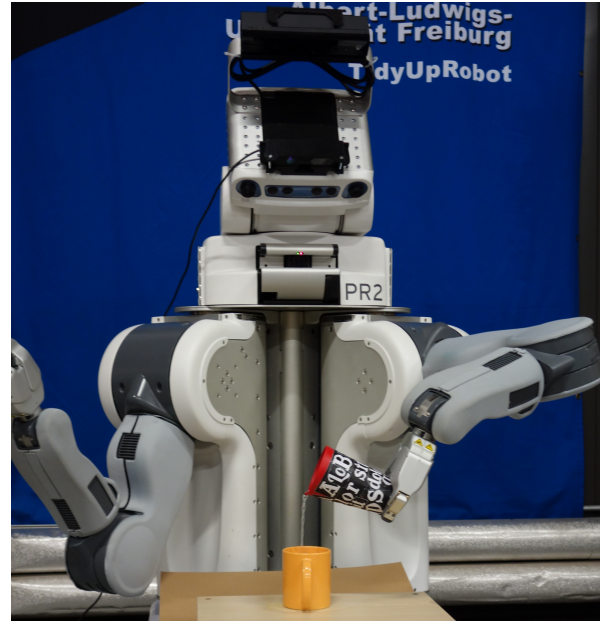


Fig. 1: The PR2 robot pouring water into a cup.

The main contributions of this paper are (i) the application of recent techniques in deep reinforcement learning to the problem of pouring specific liquid heights without spillage and collision, (ii) the definition of the pouring task as a goal directed behavior which allows the system to generalize to different targets, and (iii) the transfer of the policy learned in simulation to a real robot.

II. RELATED WORK

Our approach applies recent developments in deep policies to the application of robot pouring. We therefore provide an overview of relevant work about using deep policies for continuous control and techniques for liquid pouring with robots.

A. About Deep Policies for Continuous Control

The application of deep policies for continuous control has seen a growing interest in recent years due to their capacity to learn and model complex dynamics from real [8, 12] and/or simulated experiences [14, 10, 2], while being able to generalize to previously unseen scenarios [16].

Although there are several techniques capable of dealing with continuous action spaces, Deep Deterministic Policy Gradients (DDPG) [14] has shown impressive results when dealing with this type of problem. Moreover, a recently proposed innovation, Hindsight Experience Replay (HER) [2], has expanded the applicability and flexibility of these policies when trained on environments with sparse rewards.

Recent approaches, however, have mainly focused on learning elementary manipulation tasks like *pushing* [21], *picking* [22] and *reaching* [13]. In this paper, on the contrary, we focus on applying recent techniques to the more difficult task of pouring liquids with a robotic arm. The trained policy does not only need to learn optimal pouring trajectories, but it needs to learn the complex dynamics controlling liquid behavior.

B. About Liquid Pouring Strategies

Different aspects of liquid pouring have been investigated over the years, resulting in various approaches. A popular method is to learn the pouring trajectory from demonstration [11, 24, 30].

Focusing on the problems of collision and spillage avoidance, Pan and Manocha [18] use a simplified dynamics model based on approximations. In a follow-up paper [19], they use a neural network to learn low-dimensional features to predict the state of the liquid. In both papers, they did not test on a real robot, and assume the features for their approaches can be acquired from sensing data. Lopez-Guevara et al. [15] focus on minimizing spillage while pouring liquids into novel containers. They first calibrate an approximate fast liquid simulator using spillage information obtained from a real robot performing the pouring. They then used this calibrated simulator in deployment on a robot to determine the optimal action for spillage reduction. Tamosiunaite et al. [28] focused on simultaneous goal and shape learning for dynamic motion primitives using reinforcement learning.

In the area of pouring liquids to specific heights, Schenck and Fox [26] focus on the detection of water in images using convolutional neural networks. For pouring they control the rotation of the robot wrist using a PID controller, with the liquid level as input. In a follow-up paper [27], they look at reducing the divergence between liquid simulation and real liquid behavior.

In previous work [4], we combined the relationship outlined in Section IV-B with a PID controller for controlling the rotation of the robot wrist. In this case, a trajectory was not learned and the bottle was prepositioned with respect to the cup before employing the PID controller.

Recently, Yamaguchi and Atkeson [29] use a model-based reinforcement learning approach for pouring. They employ neural networks to learn the dynamics. In contrast, our approach applies model-free reinforcement learning, removing the need to learn a complicated model. Furthermore, they only present preliminary results on a real robot, making it difficult to ascertain how well their approach performs.

In contrast to these previous works, we focus on the problem of learning a pouring trajectory that avoids collisions and spillage and is able to pour to a specific height. We furthermore demonstrate that it is possible to learn complex liquid dynamics in simulation and then transfer it to a real robot.

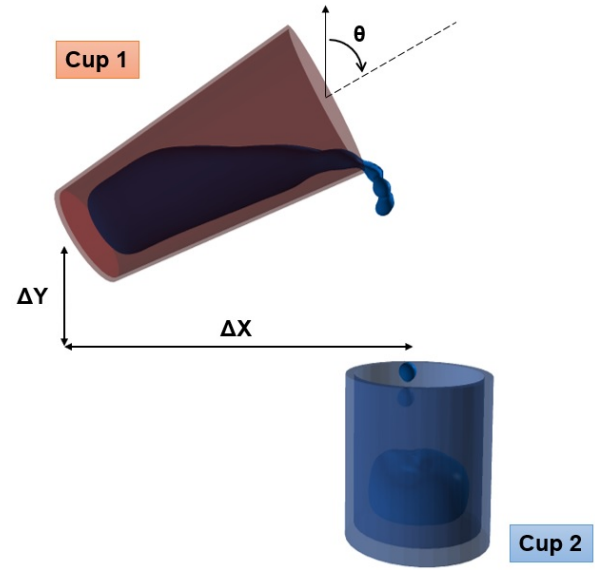


Fig. 2: PreonLab training scene. The policy controls the displacement and rotation of *Cup 1*. The simulation keeps track of the volumes inside source (*Cup 1*) and destination (*Cup 2*) containers and the amount of spillage. Liquid properties are set to replicate the behavior of water.

III. LEARNING POURING TRAJECTORIES

A. Simulation Environment

We train our policy in simulation using *PreonLab*, a state-of-the-art fluid simulator developed by FIFTY2 [6]. *PreonLab* provides highly accurate simulation results while keeping the computational cost very low, both of which are critical requirements for training our policy and transferring it to the real robot.

Figure 2 shows the simulation scene which is used for training. Both cups are modeled based on the ones which will be used in real life. The initial position of the cups is fixed, but techniques like *curriculum learning* [3] could be used to progressively explore more challenging scenarios. The liquid properties can be modified as necessary and were set to replicate the behavior of water. A boundary region (not shown in Figure 2) was defined around the scene in order to constrain the region of interest. Moving outside this area is defined as a collision. The simulation keeps track of the position of each cup, the liquid volume inside them, and the amount of liquid which has been spilled so far.

B. Reinforcement Learning Setup

We now describe the Reinforcement Learning (RL) scenario which was defined to learn the pouring policy. We also describe some of the techniques which we employed for optimizing the training process and to improve the transfer of the trained policies from simulation to the real robot.

1) *State and Action*: The current state of the environment s_t is represented as a 9D vector containing the following information: the relative distance in cm between the cups in 2D (depth is not considered), the rotation angle of the source container ($^\circ$), the fill level in *Cup 2* (%), the amount

of liquid which has been spilled (ml), the filling rate (%), and the action taken during the previous time step. Actions a_t from the policy specify a relative displacement for the source container with respect to its current position in 2D and a relative rotation in its operation plane. This yields a 3D action space. Each action is re-scaled to the ranges $\pm 1\text{cm}$ and $\pm 1^\circ$ for linear and angular displacements respectively.

2) *Reward Function and Goal Definition*: Manually shaping a reward function that correctly reflects the task at hand can not only be very challenging [17], but it also requires domain-specific knowledge. Moreover, incorrectly shaping the reward may bias training and hinder the learning process. Recently, an innovative approach was proposed to allow off-policy RL algorithms to be successfully trained using sparse rewards. Hindsight Experience Replay (HER) [2] encourages off-policy algorithms to make better use of the experiences collected so far by manipulating their replay buffer. Typically, after finishing one episode, every transition $s_t \rightarrow s_{t+1}$ is stored in memory together with their respective rewards and the goal for that particular episode. With HER, however, additional experiences are generated by storing the collected transitions with different goals. These goals are state configurations which were effectively reached during the episode and which will therefore generate positive rewards for some of the “augmented” transitions. Given that the goal of an episode does not influence the dynamics of the environment, these “augmented” experiences will remain valid and will allow the algorithm to learn not only the reward function, but also the purpose of the task and the dynamics of the environment.

In this paper we are interested in training a policy capable of pouring different target volumes while minimizing spillage. In order for the policy to generalize to different targets, however, it must receive the current goal as an input [25]. Although it is tempting to define the goal for an episode just as the amount of liquid to pour while penalizing spillage, we noticed that including the amount of spilled liquid in our goal representation speeds up the training process. By defining our goal as a 2D vector of the expected fill level (%) and spillage (ml) during an episode, we encourage the policy to learn the dynamics of the liquid also from sub-optimal transitions (i.e., those where the liquid is spilled).

Following the rewards definition proposed by Schaul et al. [25]: $r(s_t, a, g) = [d(s_t, g) < \epsilon]$, our agent will get a positive reward r when the distance d between the current state and the provided goal g is less than ϵ , and a negative reward (*step cost*) otherwise. We apply an additional penalty when a collision is detected. During training we set ϵ so that the goal is reached if the fill level is within 10% of the target and spillage is less than 25ml.

3) *Domain Randomization*: Domain randomization has proven quite useful when transferring policies from simulation into the real world by introducing several sources of noise during the simulated training [1, 21, 22]. In our experiments, and similar to the approach proposed in [21], we model the observation noise which is expected from the

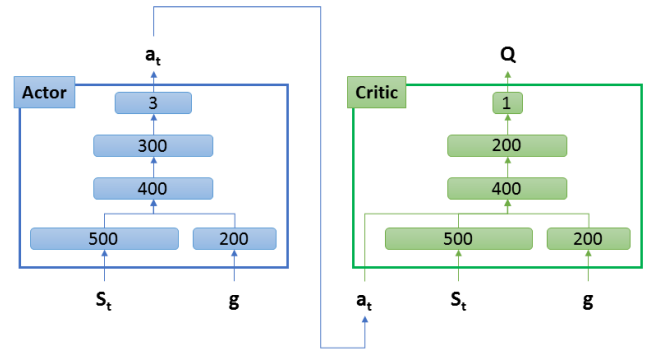


Fig. 3: Schematic of the policy (*actor*) and the value (*critic*) networks. ELU activations are used after each hidden layer. The output of the critic consists of linear units, while the output of the actor employs a $\tanh$ function to bound the policy actions.

sensors in the robot as independent Gaussian noise applied to each state feature. Training our agent with noisy observations encourages the system to learn a robust policy capable of dealing with the uncertainty expected in the real world.

C. Training

In this paper we use DDPG [14], an *actor-critic* reinforcement learning algorithm capable of learning deterministic continuous action policies. DDPG maintains two neural networks also known as the *actor* and the *critic*. The employed architectures for both of these networks are shown in Figure 3. The *actor network* takes the current state s_t and the episode goal g as inputs and outputs the action a_t which should be executed. The *critic* takes the selected action a_t as an additional input and evaluates the utility of that action given the current state and goal.

In order to substantially improve the data-efficiency of the algorithm, we employ the asynchronous version proposed by [23]. This asynchronous version of DDPG allows us to distribute the training of the networks and the data collection over several computers. In our case, training is conducted over two computers in parallel.

During training, the target volumes to pour are randomly generated between 20% and 100% for each episode. The target spillage, however, is always set to zero. The HER module will take care of augmenting the replay memory with transitions where spilling was actually desired.

Collisions are detected and avoided before executing the action in the simulator. Any action which would result in the cups colliding or *Cup 1* leaving the operation area are penalized and stored in the replay memory without changing the original state.

Another important factor to consider when implementing the policy in the real robot is the expected latency between perception and control which is usually neglected during simulation. This latency can be modeled as uncertainty in the time step between actions. Although we would like the agent to be able to select and execute actions as fast as possible in order to achieve high accuracy in its movements, we need

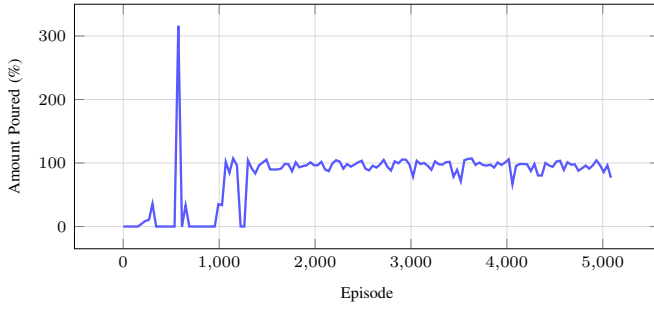


Fig. 4: Percentage of the target volume poured when validating the policy as training progresses. Targets are randomly sampled between 20% and 100% of capacity of the destination container.

to choose an action frequency which is also attainable in the real system. By training our policy with a very large time step between actions we encourage it to learn “safe” trajectories where small pouring rates are preferred. These types of policies are usually easier to implement on real systems where the time it takes for the robot to execute an action can easily be made longer rather than shorter. Our final policy was trained with an action frequency of 1Hz, which was determined based on experiments with the real system. We did experiment with a higher action frequency of 2.5Hz, but found on the real system it was extremely difficult to operate at this frequency. It should also be considered that an amount of time is required to execute the action on the robot. Moving too fast would result in jerky movements, possibly causing spillage.

During training, the state representation s_t contains information about spillage. Collecting this information in the real system, however, can be challenging and inaccurate. When running experiments on the real robot, we use the fact that the policy has learned not to spill in order to avoid feeding this information back into the state. By setting spillage to zero during evaluation we can still observe how the policy behaves when a goal involving only pouring is given.

Figures 4 and 5 show the performance of the policy as the number of experienced episodes increases. One episode lasts 30s and this value was tuned in order to give the policy enough time to achieve the task under the given conditions. Figure 4 clearly shows that the policy learns to pour the target volume even when working with noisy measurements. Moreover, we can see from Figure 5 that the policy has also learned to avoid spilling.

IV. IMPLEMENTATION DETAILS

In order to execute the learned policy on a real robot, the state information described in Section III-B.1 must be obtained from the environment. Specifically, the relative cup distances, the rotation angle of *Cup 1* and the fill level in *Cup 2* must be determined from perception. During operation, the spillage is assumed to be zero and the remaining state values can be easily obtained. Furthermore, the errors in the perception should be minimized as much as possible and obtaining the state information should be within the action

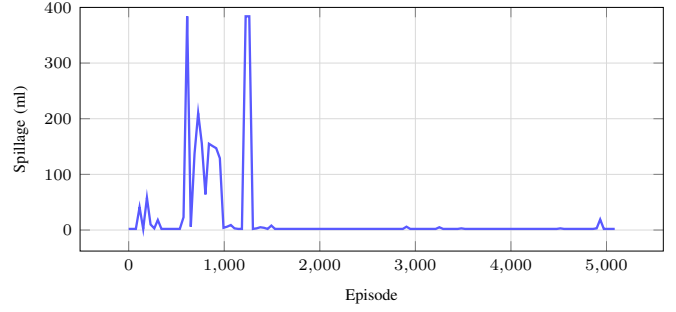


Fig. 5: Spillage volume when validating the policy during training.

frequency the policy was trained with. In the next sections we described these issues in detail.

A. Cup Distances

To determine the relative distances of the cups, it is necessary to know the pose of each cup. When *Cup 1* is placed in the gripper of the robot, it is firmly grasped and the transform relative to the gripper is assumed to be static. Then SimTrack [20] is used to determine the transform between the cup and gripper. The known joint positions of the robot arm is then used in combination with forward kinematics to determine its pose throughout the pour. This proved to be more accurate than tracking *Cup 1* with SimTrack.

The pose of *Cup 2* is determined at the beginning of the pour, by finding the plane corresponding to the table in the point cloud and extracting all points above it. We then determine the location of the cylinder corresponding to the cup using RANSAC [7]. The pose of *Cup 2* is assumed to remain static throughout the pour.

B. Fill Level Detection

To extract the fill level, we use the pose of *Cup 2* to determine a section of the point cloud that is within the cup boundaries. We then extract the liquid height from this section. We obtained the depth information with an ASUS Xtion RGB-D camera mounted on the robot and looking into the cup. For opaque liquids such as milk and orange juice, we directly take the extracted liquid height as the real liquid height. However, for transparent liquids such as water and apple juice, the infrared light from the camera is *refracted* resulting in an incorrectly measured liquid height. To handle this, we employ the following relationship

$$h = \left(\frac{\sqrt{n_l^2 - 1 + \cos^2(\alpha)}}{\sqrt{n_l^2 - 1 + \cos^2(\alpha)} - \cos(\alpha)} \right) h_r. \quad (1)$$

Here h refers to the estimated fill level and h_r is the raw height measured from the point cloud. The term n_l is the index of refraction of the liquid and α is the incidence angle of infrared light from the camera projector with respect to the normal of the liquid. Further details regarding this relationship can be found in the works of Hara et al. [9] and Do et al. [5]. In order to track and reduce the noise in the measurements, we apply a Kalman filter for both the transparent and opaque liquids.

V. EXPERIMENTAL RESULTS

We evaluated the learned policy in simulation and in real experiments using a PR2 robot. In both cases, we look at how the policy performs pouring to different heights with water, which it was trained for. In the experiments carried out with our PR2 robot, we additionally analyze how well the policy generalizes to other liquids.

A. Simulation Results

Table I shows the performance of the policy in simulation under noisy observations. The policy was given a target volume to pour while avoiding spillage. The first aspect to notice is that the system learned to avoid collisions both with the second cup and with the boundaries of the scene. Moreover, the spillage in all our experiments is minimal even in the presence of noise. Regarding the pouring level, it is clear that our goal margin ϵ plays an important role in the accuracy we get from the policy. Given that the goal is considered to be reached if within 10% of the target level, it seems that the policy has learned to pour just enough to reach the goal while accounting for any possible noise in the measurements. Because of this, the policy usually undershoots the target.

TABLE I: Pouring results in simulation averaged over 10 runs

Target (%)	Poured Level (%)	Spillage (ml)	Collisions
80	78.32 ± 5.62	0.3 ± 0.48	0
60	55.07 ± 0.71	0	0
40	37.46 ± 2.86	0	0
20	18.96 ± 2.22	0.1 ± 0.3	0

B. Real Robot Experiments

We first evaluate how the policy performs when pouring water to different heights. For each height, we performed ten pours and in each pour, *Cup 1* was filled with approximately 400ml. We measured the spilled liquid using a scale and the final liquid height with a ruler. The results can be seen in Table II. In general, they are comparable to the simulation results, implying that it is possible to deploy the learned policy on a real robot.

However, there are some differences. As can be seen, there is more spillage as the target height increases. During the experiments, we noticed that the fill level is overestimated when there is more liquid present in *Cup 2*. We believe this is due to more disturbance in the liquid surface, resulting in inaccurate detection from the point cloud. Furthermore, Eq. (1) magnifies the amount of noise in the raw measurement. As a result of this, the policy would stop pouring, assuming it had reached the desired level. After a few moments, the fill level detection would stabilize, providing a lower fill level and the policy would then rotate back to pour more. This rotation back and forth resulted in more spillage, mainly when the liquid clings to the cup and rolls down it. This behavior was not observed in the simulation and suggests there are differences between simulation and real life.

TABLE II: PR2 results, mean and standard deviation over 10 runs

Target (%)	Poured Level (%)	Spillage (ml)	Collisions
80	72.70 ± 4.40	3.4 ± 2.67	0
60	55.96 ± 4.39	4.1 ± 4.15	0
40	35.11 ± 2.87	0.8 ± 1.93	0
20	14.49 ± 2.45	0	0

To give some perspective of the pouring accuracy results, we convert the averages over all the pours to a volume error, giving us a value of 19.96ml over 40 pours. In their paper, Schenck and Fox [26] report an average volume error of 38ml over 30 pours. In our previous work [4], for a similar cup, we achieved a pouring error of 23.9ml over 10 pours. For spillage, Lopez-Guevara et al. [15] report a mean spillage percent (of the initial liquid mass) of around 7%. This was achieved iteratively on the real robot, where the spillage amount was provided after each iteration for optimizing the robot action.

An interesting question is whether or not the same learned policy can be applied to different liquids. We hypothesize that if the dynamics of the liquids are similar, this should be possible. To investigate this, we evaluated the same policy on milk, orange juice, apple juice and olive oil. For each of these liquids we poured to a target height of 50% and performed ten pours with the PR2. The results can be seen in Table III.

TABLE III: Different liquids, mean and standard deviation over 10 runs. No collisions occurred.

Liquid (50 % target)	Poured Level (%)	Spillage (ml)
Milk	38.43 ± 1.38	5.6 ± 11.83
Orange juice	40.49 ± 1.30	2.10 ± 2.13
Apple juice	45.96 ± 4.48	6.1 ± 4.7

As can be seen from the table, the policy learned in simulation can be transferred to multiple liquids, which demonstrates its robustness. Interestingly, the poured levels for milk and orange juice (both opaque liquids) show a higher error than those for the transparent liquids. We found that this is due to lower noise in the fill level estimation. As noted before, we take the raw height from the point cloud to estimate the height of opaque liquids. The resulting measurements have lower noise and as a result, the robot only pours once to reach the target height. The policy was trained with a successful pour defined as being within 10% of the target height. Therefore, when the fill level was within this range, it stopped pouring. Additionally, despite only pouring with one turn, the spillage for milk is still comparable to transparent liquids. This is due to the fact that milk had a higher tendency to run along the edge of the cup during the initial rotation. One possible solution could be to modify the cohesion parameters of the liquid during training. Only in the case of olive oil, the policy failed to pour correctly. For each attempt, the oil ran along the edge of the cup and spilled. Interestingly, a previous policy trained with a different liquid

model learned to pour by moving *Cup 1* such that the spilled liquid would fall into *Cup 2*.

VI. CONCLUSIONS

In this paper, we applied recent advances in deep reinforcement learning to the problem of pouring liquids with a robot to specific heights while avoiding spillage and collisions. Our policy was trained solely in a state-of-the-art liquid simulator which provided the ability to learn complex fluid dynamics. Our experimental results demonstrate that we were able to successfully transfer the learned policy to a real robot thereby enabling it to robustly pour liquids. We furthermore demonstrated that although the policy was trained on water and in simulation, it could be successfully applied to several other liquids without the need for retraining.

REFERENCES

- [1] Joshua Tobin and Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. 2017. URL <http://arxiv.org/abs/1703.06907>.
- [2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. 2017. URL <http://arxiv.org/abs/1707.01495>.
- [3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML '09*, pages 41–48, New York, NY, USA, 2009.
- [4] Chau Do and Wolfram Burgard. Accurate pouring with an autonomous robot using an RGB-D camera. In *International Conference on Intelligent Autonomous Systems (IAS)*, Baden Baden, Germany, 2018.
- [5] Chau Do, Tobias Schubert, and Wolfram Burgard. A probabilistic approach to liquid level detection in cups using an RGB-D camera. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Daejeon, Korea, 2016.
- [6] FIFTY2 Technology GmbH. PreonLab. www.fifty2.eu.
- [7] Martin A Fischler and Robert C Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [8] S. Gu, E. Holly, T. Lillicrap, and S. Levine. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3389–3396, May 2017.
- [9] Yoshitaka Hara, Fuhito Honda, Takashi Tsubouchi, and Akihisa Ohya. Detection of Liquids in Cups Based on the Refraction of Light with a Depth Camera Using Triangulation. 2014.
- [10] Nicolas Heess, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, Tom Erez, Ziyu Wang, S. M. Ali Eslami, Martin A. Riedmiller, and David Silver. Emergence of locomotion behaviours in rich environments. *CoRR*, abs/1707.02286, 2017. URL <http://arxiv.org/abs/1707.02286>.
- [11] Joshua D Langsfeld, Krishnanand N Kaipa, Rodolphe J Gentili, James A Reggia, and Satyandra K Gupta. Incorporating failure-to-success transitions in imitation learning for a dynamic pouring task. In *Workshop on Compliant Manipulation: Challenges and Control, Chicago, IL*, 2014.
- [12] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *J. Mach. Learn. Res.*, 17(1):1334–1373, January 2016.
- [13] A. Levy, R. Platt, and K. Saenko. Hierarchical Actor-Critic. *ArXiv e-prints*, Dec 2017. URL <https://arxiv.org/abs/1712.00948>.
- [14] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *ICLR*, 2016.
- [15] Tatiana Lopez-Guevara, Nicholas K Taylor, Michael U Gutmann, Subramanian Ramamoorthy, and Kartic Subr. Adaptable pouring: Teaching robots not to spill using fast but approximate fluid simulation. In *Conference on Robot Learning*, pages 77–86, 2017.
- [16] Yan Mengyuan, Frosio Iuri, Tyree Stephen, and Kautz Jan. Sim-to-real transfer of accurate grasping with eye-in-hand observations and continuous control. 2017. URL <https://arxiv.org/abs/1712.03303>.
- [17] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning, ICML*, pages 278–287, 1999.
- [18] Zherong Pan and Dinesh Manocha. Motion planning for fluid manipulation using simplified dynamics. In *Intelligent Robots and Systems (IROS), 2016 IEEE/RSJ International Conference on*, 2016.
- [19] Zherong Pan and Dinesh Manocha. Feedback motion planning for liquid pouring using supervised learning. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2017, Vancouver, BC, Canada, September 24-28, 2017*, pages 1252–1259, 2017.
- [20] Karl Pauwels and Danica Kragic. Simtrack: A simulation-based framework for scalable real-time object pose detection and tracking. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1300–1307. IEEE, 2015.
- [21] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. 2017. URL <http://arxiv.org/abs/1710.06537>.
- [22] Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. 2017. URL <http://arxiv.org/abs/1710.06542>.
- [23] Iyaylo Popov, Nicolas Heess, Timothy P. Lillicrap, Roland Hafner, Gabriel Barth-Maron, Matej Vecerik, Thomas Lampe, Yuval Tassa, Tom Erez, and Martin A. Riedmiller. Data-efficient deep reinforcement learning for dexterous manipulation. *CoRR*, 2017. URL <http://arxiv.org/abs/1704.03073>.
- [24] Leonel Rozo, Pablo Jiménez, and Carme Torras. Force-based robot learning of pouring skills using parametric hidden markov models. In *Robot Motion and Control (RoMoCo), 2013 9th Workshop on*, pages 227–232. IEEE, 2013.
- [25] Tom Schaul, Dan Horgan, Karol Gregor, and David Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning - Volume 37, ICML*, pages 1312–1320, 2015.
- [26] Connor Schenck and Dieter Fox. Visual closed-loop control for pouring liquids. In *Robotics and Automation (ICRA), 2017 IEEE International Conference on*, pages 2629–2636. IEEE, 2017.
- [27] Connor Schenck and Dieter Fox. Reasoning about liquids via closed-loop simulation. *arXiv preprint arXiv:1703.01656*, 2017.
- [28] Minija Tamosiunaite, Bojan Nemec, Aleš Ude, and Florentin Wörgötter. Learning to pour with a robot arm combining goal and shape learning for dynamic movement primitives. *Robotics and Autonomous Systems*, 59(11):910–922, 2011.
- [29] Akihiko Yamaguchi and Christopher G Atkeson. Neural networks and differential dynamic programming for reinforcement learning problems. In *Robotics and Automation (ICRA), 2016 IEEE International Conference on*, pages 5434–5441. IEEE, 2016.
- [30] Akihiko Yamaguchi, Christopher G Atkeson, and Tsukasa Ogasawara. Pouring skills with planning and learning modeled from human demonstrations. *International Journal of Humanoid Robotics*, 12(03): 1550030, 2015.